

Topic: Unit 1

BCA 6TH SEM DS2 ML

Prepared By:
Dr Tejinder Kaur
Associate Professor



ML

Machine learning (ML) allows computers to learn and make decisions without being explicitly programmed. It involves feeding data into algorithms to identify patterns and make predictions on new data. It is used in various applications like image recognition, speech processing, language translation, recommender systems, etc. In this article, we will see more about ML and its core concepts.



continue

- **Machine Learning (ML)** is a subset of artificial intelligence (AI) that focuses on enabling machines to learn from data and improve their performance over time without being explicitly programmed for each task. It involves the use of algorithms and statistical models to analyze and draw inferences from patterns in data, allowing computers to make predictions or decisions based on new data.
- **Key Principles of Machine Learning**
- **Decision Process:** Machine learning algorithms are designed to make predictions or classifications based on input data. This data can be labeled (supervised learning) or unlabeled (unsupervised learning).
- **Error Function:** This function evaluates the accuracy of the model's predictions. It compares the predicted values with the actual values to determine the error.
- **Model Optimization:** The algorithm iteratively adjusts its parameters to minimize the error and improve accuracy. This process continues until the model reaches a satisfactory level of performance.

Continue

Introduction To Machine Learning

Data



Training the
Machine



Build a Model



Predicting
Outcome





Continue

- Machine Learning is mainly divided into three core types: Supervised, Unsupervised and Reinforcement Learning along with two additional types, Semi-Supervised and Self-Supervised Learning.
- **Supervised Learning**: Trains models on labeled data to predict or classify new, unseen data.
- **Unsupervised Learning**: Finds patterns or groups in unlabeled data, like clustering or dimensionality reduction.
- **Reinforcement Learning**: Learns through trial and error to maximize rewards, ideal for decision-making tasks.



Continue

- **Self-Supervised Learning**: Self-supervised learning is often considered as a subset of unsupervised learning, but it has grown into its own field due to its success in training large-scale models. It generates its own labels from the data, without any manual labeling.
- **Semi-Supervised Learning**: This approach combines a small amount of labeled data with a large amount of unlabeled data. It's useful when labeling data is expensive or time-consuming.



CONTINUE

- **Module 1: Machine Learning Pipeline**
- This section covers preprocessing, exploratory data analysis and model evaluation to prepare data, uncover insights and build reliable models.
- **1. Data Preprocessing**
- ML workflow
- Data Cleaning
- Data Preprocessing in Python
- Feature Scaling
- Feature Extraction
- Feature Engineering
- Feature Selection Techniques



Continue

- **2. Exploratory Data Analysis**
- Exploratory Data Analysis
- Exploratory Data Analysis in Python
- Advance EDA
- Time Series Data Visualization



Continue

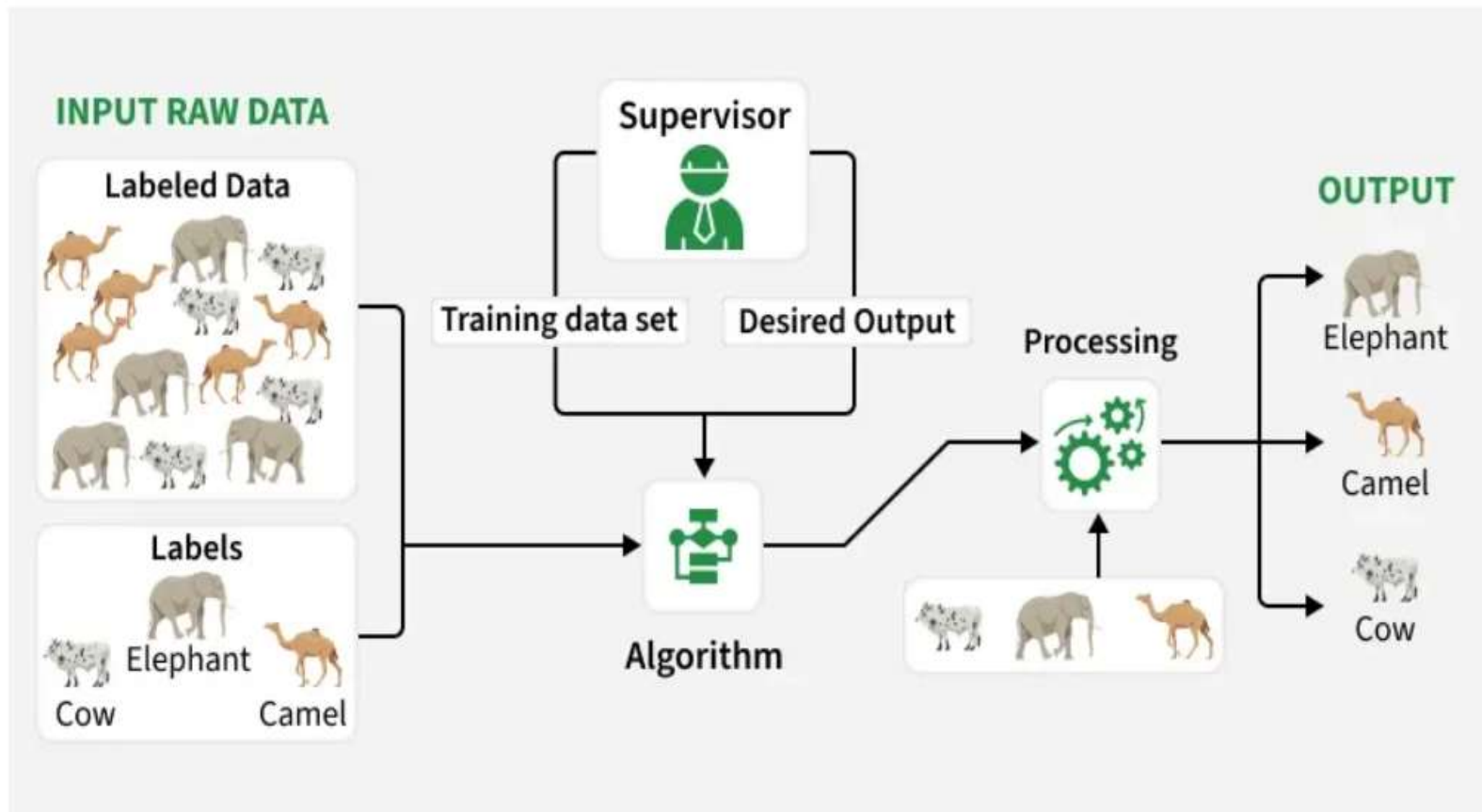
- **3. Model Evaluation**
- Regularization in Machine Learning
- Confusion Matrix
- Precision, Recall and F1-Score
- AUC-ROC Curve
- Cross-validation
- Hyperparameter Tuning



Continue

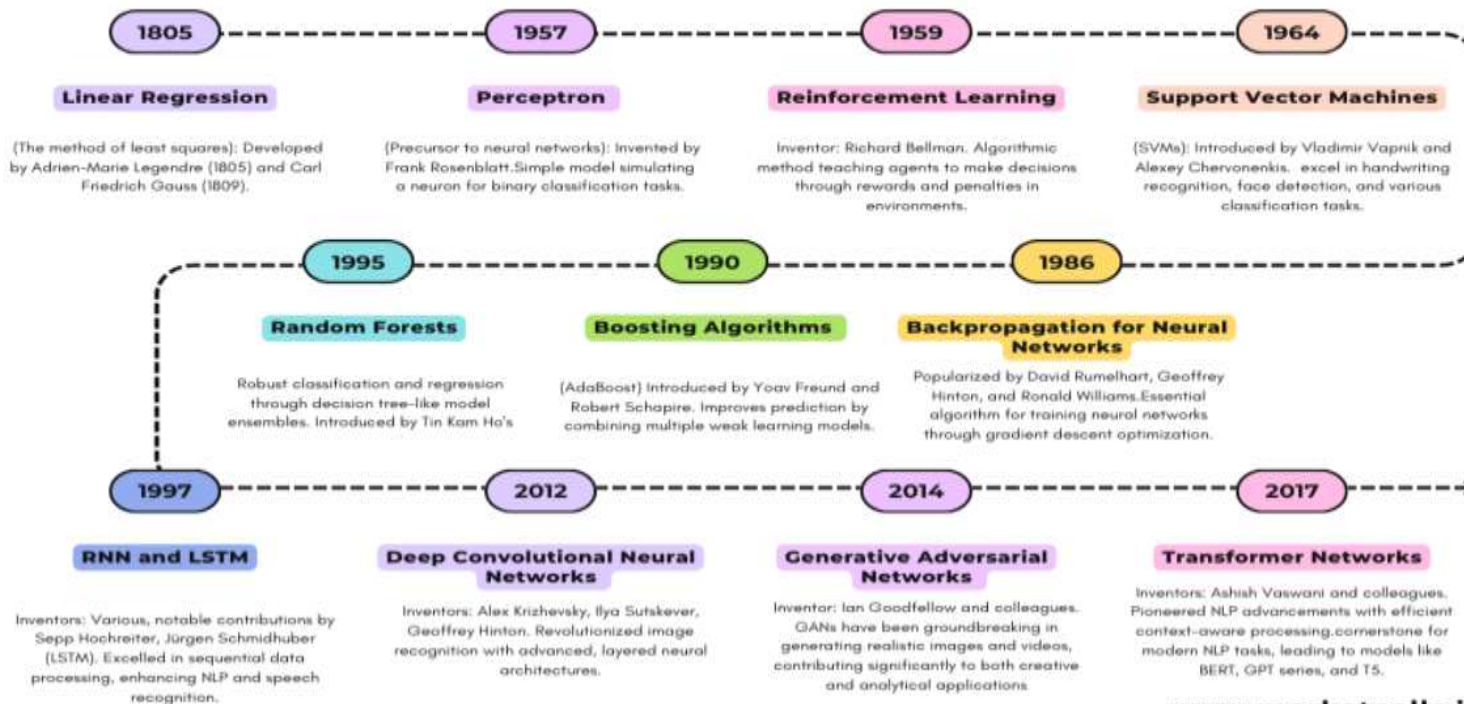
- **Module 2: Supervised Learning**
- Supervised learning algorithms are generally categorized into **two main types**:
- **Classification** - where the goal is to predict discrete labels or categories
- **Regression** - where the aim is to predict continuous numerical values.

Continue



History ML

Evolution of Machine Learning





features of ML

- **Learning from Data**

ML systems automatically learn patterns and relationships from historical or real-time data without being explicitly programmed.

- **Adaptability & Self-Improvement**

Models improve their performance over time as more data becomes available.

- **Automation of Decision-Making**

ML enables automated predictions, classifications, and decisions with minimal human intervention.

- **Pattern Recognition**

Efficiently identifies hidden patterns, trends, and correlations in large and complex datasets.

- **Scalability**

Capable of handling massive volumes of structured and unstructured data (big data).

- **Generalization Capability**

Learns from training data and applies knowledge to unseen or new data.



CONTINUE

- Accuracy & Precision**

Provides high accuracy in predictions when trained with quality data and appropriate algorithms.

- Real-Time Processing**

Supports real-time or near real-time analytics (e.g., fraud detection, recommendation systems).

- Flexibility Across Domains**

Applicable in diverse fields such as healthcare, finance, smart cities, IoT, cybersecurity, and education.

- Reduced Human Bias**

Decisions are based on data-driven insights rather than subjective human judgment.

- Feature Extraction & Selection**

Automatically identifies the most relevant attributes from raw data.

- Continuous Optimization**

Uses optimization techniques to minimize error and enhance model performance.



working of ML

- **Data Collection**

Relevant data is gathered from various sources such as databases, sensors, IoT devices, logs, or user inputs.

- **Data Preprocessing**

The collected data is cleaned and prepared:

- Removing noise and missing values
- Normalization and scaling
- Feature selection and transformation



CONTINUE

- **Feature Extraction**

Important features (attributes) are identified from raw data to improve learning efficiency and accuracy.

- **Model Selection**

An appropriate ML algorithm is chosen based on the problem type:

- **Supervised Learning** (classification, regression)
- **Unsupervised Learning** (clustering, association)
- **Reinforcement Learning** (decision-making through rewards)



CONTINUE

- Training the Model**

The algorithm learns patterns by analyzing training data and adjusting parameters to minimize errors.

- Model Evaluation**

The trained model is tested using unseen data to measure performance using metrics like accuracy, precision, recall, RMSE, etc.

- Prediction / Decision Making**

Once validated, the model is used to make predictions or decisions on new real-world data.

- Deployment**

The trained model is integrated into applications, systems, or services.

- Feedback & Improvement**

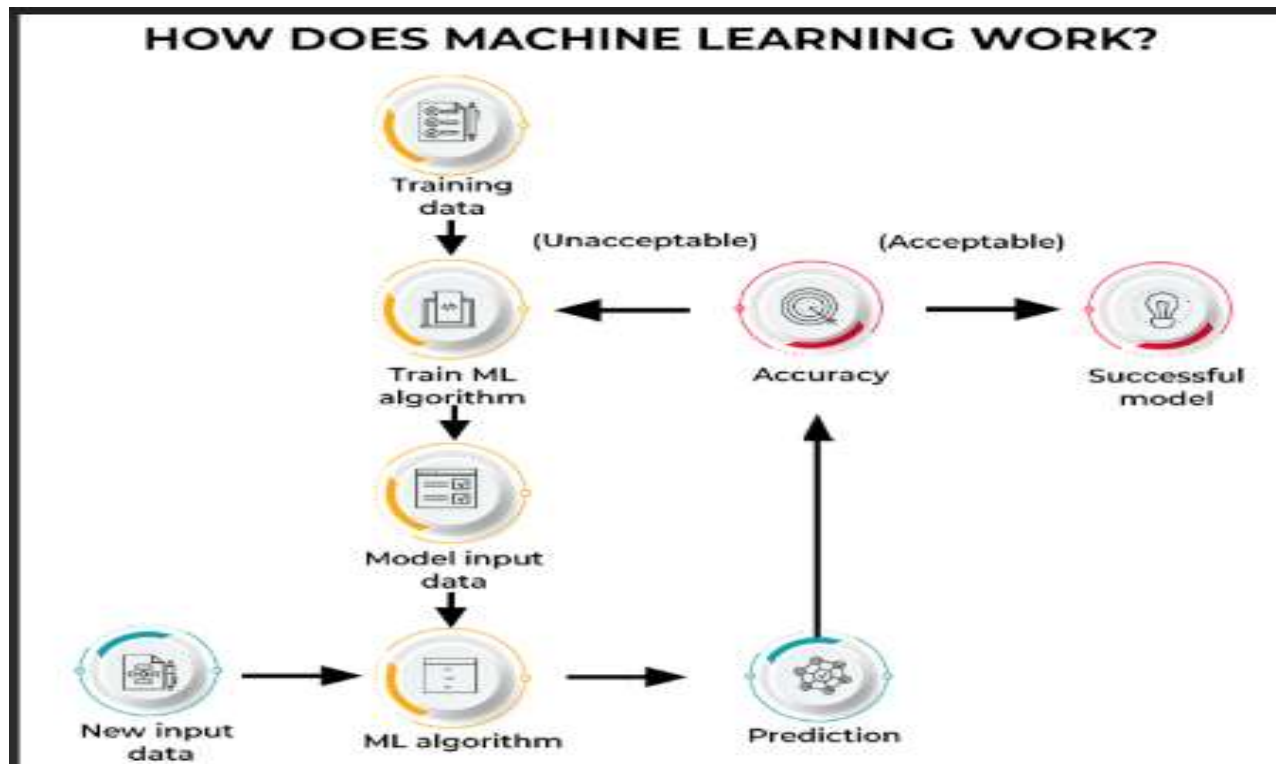
Model performance is continuously monitored and updated using new data for better accuracy.



CONTINUE

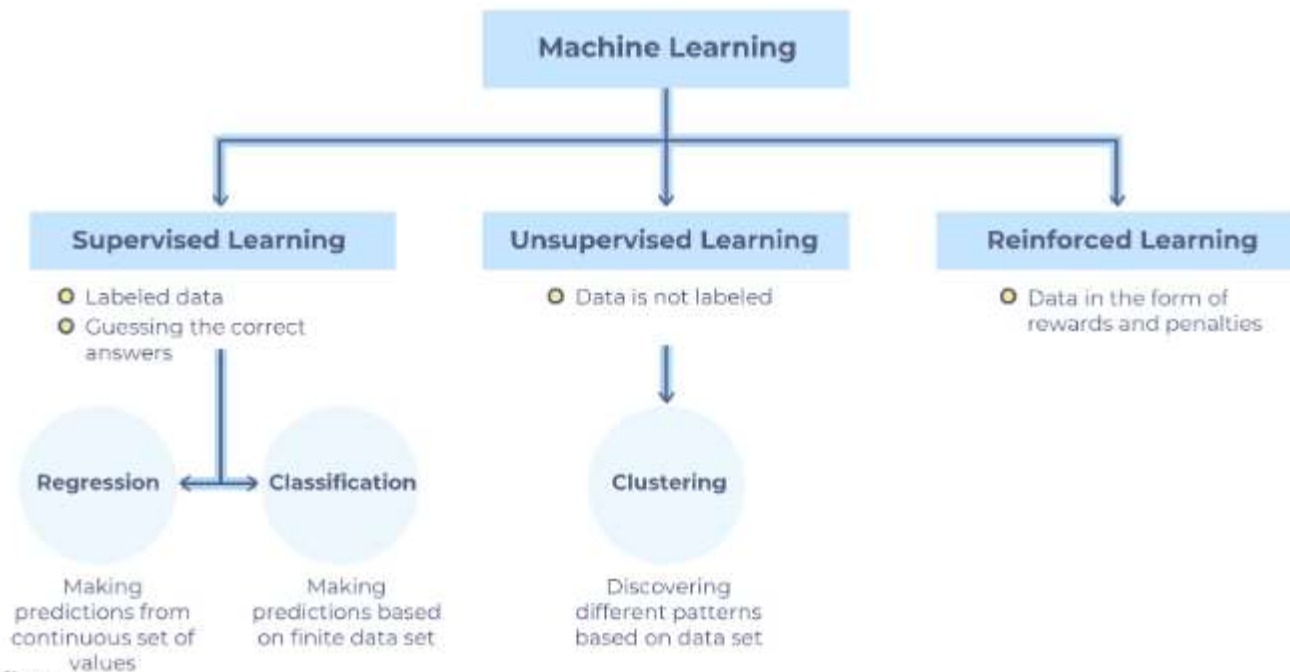
- **Simple Working Flow:**
- **Data → Preprocessing → Feature Extraction → Model Training → Evaluation → Prediction → Improvement**

CONTINUE



Classification of ML.

ML CLASSIFICATION





CONTINUE

- **1. Supervised Learning**
- The model is trained using **labelled data** (input with known output).
- **Key Characteristics:**
- Requires input–output pairs
- High accuracy for prediction tasks
- Used for classification and regression
- **Common Algorithms:**
- Linear Regression
- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- k-Nearest Neighbor (k-NN)
- Naïve Bayes
- **Applications:**
- Email spam detection
- Disease diagnosis
- Image and speech recognition



CONTINUE

- **2. Unsupervised Learning**
- The model learns from **unlabeled data** and discovers hidden patterns.
- **Key Characteristics:**
- No predefined output
- Data grouping and pattern discovery
- **Common Algorithms:**
- K-Means Clustering
- Hierarchical Clustering
- DBSCAN
- Principal Component Analysis (PCA)
- Apriori Algorithm
- **Applications:**
- Customer segmentation
- Market basket analysis
- Anomaly detection



CONTINUE

- **4. Reinforcement Learning**
- The model learns by **interacting with an environment** and receiving **rewards or penalties**.
- **Key Characteristics:**
- Trial-and-error learning
- Focuses on sequential decision-making
- **Common Algorithms:**
- Q-Learning
- SARSA
- Deep Q-Network (DQN)
- Policy Gradient Methods
- **Applications:**
- Robotics
- Game playing (AlphaGo)
- Autonomous vehicles



CONTINUE

- **5. Deep Learning (Sub-class of ML)**
- Uses **multi-layer neural networks** for complex learning tasks.
- **Key Characteristics:**
- Handles large-scale and unstructured data
- Automatic feature extraction
- **Common Models:**
- Artificial Neural Networks (ANN)
- Convolutional Neural Networks (CNN)
- Recurrent Neural Networks (RNN)
- Transformers
- **Applications:**
- Image and video analysis
- Natural Language Processing
- Speech recognition



CONTINUE

- **Simple Classification Diagram (Textual):**
- **Machine Learning → Supervised | Unsupervised | Semi-Supervised | Reinforcement | Deep Learning**

Data Pre-Processing

- **Steps in Data Pre-Processing**
- **Data Collection**
 - Gather data from databases, sensors, IoT devices, surveys, logs, etc.
- **Data Cleaning**
 - Handling missing values (mean, median, mode, deletion)
 - Removing noise and outliers
 - Correcting inconsistent or duplicate data
- **Data Integration**
 - Combining data from multiple sources into a single dataset
 - Resolving data conflicts and redundancies
- **Data Transformation**
 - Normalization (Min-Max Scaling)
 - Standardization (Z-score)
 - Encoding categorical data (Label encoding, One-Hot encoding)
 - Log or power transformations

CONTINUE

- **Data Reduction**
 - Feature selection
 - Feature extraction (PCA)
 - Dimensionality reduction
 - Sampling techniques
- **Data Discretization**
 - Converting continuous data into discrete intervals (binning)
- **Handling Imbalanced Data**
 - Oversampling (SMOTE)
 - Undersampling
 - Cost-sensitive learning



CSV File

- **CSV File (Comma-Separated Values)**

A **CSV (Comma-Separated Values)** file is a simple, widely used text file format for storing **tabular data**, where each line represents a row and values are separated by commas.



CONTINUE

- **Features of CSV File**
- Plain text format
- Easy to read and edit
- Lightweight and fast
- Supported by almost all software tools
- Platform independent



Structure of a CSV File

Name,Age,Department,Salary

Anil,28,IT,45000

Ravi,32,HR,38000

Sneha,26,Finance,42000



CONTINUE

First row → Column headers

- **Each next row** → Data records

- **Comma (,)** → Separator (sometimes semicolon ;)



Data Frame Basics

A DataFrame in Python's pandas library is a two-dimensional labeled data structure used for data manipulation and analysis. It can handle different data types such as integers, floats, and strings, and each column has a unique label. DataFrames are used in machine learning tasks to manipulate and analyze datasets in large sizes. They support operations such as filtering, sorting, merging, grouping, and transforming data. DataFrames can be created from various data sources, including lists, dictionaries, or external files like CSV or Excel



CONTINUE

```
pandas.DataFrame( data, index, columns, dtype, copy)
```



CONTINUE

- **data, index, columns, dtype, copy** are its parameters –
- **data** can contain lists, maps, dict, ndarray, etc.
- **index** contains the indexing values for the frames
- **columns** provide column labels for the resulting frame
- **dtype** is the data type for each column
- **copy** is for copying of data



CONTINUE

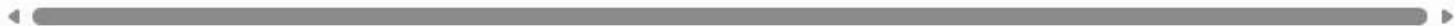
- A DataFrame is a two-dimensional labeled data structure, containing heterogeneous data. The data is arranged in a tabular format – with both a row and a column index.
- DataFrame is, in fact, the most widely used data structure of Pandas. It can be visualized as a spreadsheet or a collection of series, for example,



CONTINUE

DataFrame is, in fact, the most widely used data structure of Pandas. It can be visualized as a spreadsheet or a collection of series, for example,

Student_Name	Age	Marks	Grade
Tom	19	75.5	C
Maria	20	83.5	B
John	18	92.0	A



This dataset is represented in rows and columns, with attributes such as **Student_Name**, **Age**, **Marks**, and **Grade**.

The columns of a DataFrame can be of different data types. For example, in the above table **Student_Name** is a *String*, **Age** is an *Integer*, **Marks** is *Float*, and **Grade** is a *String* data type.

DataFrame is size mutable as well as data mutable.

Libraries for Pre-processing

- **General Data Pre-processing (Tabular / Structured Data)**
- **NumPy**
Numerical operations, arrays, math functions.
- **Pandas**
Data cleaning, handling missing values, filtering, joins, time series.
- **SciPy**
Statistical functions, normalization, transformations.
- **scikit-learn (sklearn.preprocessing)**
Scaling, encoding, imputation, feature selection (e.g., StandardScaler, OneHotEncoder, LabelEncoder, SimpleImputer)



CONTINUE

- **Natural Language Processing (Text Pre-processing)**
- **NLTK**
Tokenization, stemming, stopwords, text normalization.
- **spaCy**
Fast tokenization, lemmatization, POS tagging, pipelines.
- **Gensim**
Text cleaning, vectorization, topic modeling.
- **Hugging Face Transformers (tokenizers)**
Subword tokenization for modern NLP models.



CONTINUE

- **Image Pre-processing**
- **OpenCV (cv2)**
Resizing, normalization, filtering, augmentation.
- **Pillow (PIL)**
Basic image loading and transformations.
- **torchvision.transforms**
Image normalization and augmentation for PyTorch.
- **Albumentations**
High-performance image augmentation.



CONTINUE

- **End-to-End Data Validation & Cleaning**
- **Great Expectations**
Data quality checks and validation.
- **Pandera**
Schema-based validation for Pandas DataFrames.



Handling Missing data

- Missing values appear when some entries in a dataset are left blank, marked as NaN, None or special strings like "Unknown". If not handled properly, they can reduce accuracy, create bias and break algorithms that require complete data.
- Keeps models accurate
- Prevents bias
- Maintains sample size
- Allows correct use of ML algorithms



CONTINUE

- **Reasons Behind Missing Values in the Dataset**
- Data can be missing from a dataset for several reasons and understanding the cause is important for selecting the most effective way to handle it. Common reasons for missing data include:
 - **Technical issues:** Failed data collection or errors during data transmission.
 - **Human errors:** Mistakes like incorrect data entry or oversights during data processing.
 - **Privacy concerns:** Missing sensitive or personal information due to confidentiality policies.
 - **Data processing issues:** Errors that occur during data preparation.



CONTINUE

- **Types of Missing Values**
- **MCAR (Missing Completely at Random):** Missingness occurs randomly and is not related to any variable in the dataset.
- **MAR (Missing at Random):** Missingness depends on other observed variables, not on the missing value itself.
- **MNAR (Missing Not at Random):** Missingness is directly related to the value that is missing (e.g., high-income individuals not reporting income).



Encoding Categorical data

Categorical data refers to variables that belong to distinct categories such as labels, names or types. Since most machine learning algorithms require numerical inputs, encoding categorical data to numerical data becomes important. Proper encoding ensures that models can interpret categorical variables effectively, leading to improved predictive accuracy and reduced bias



CONTINUE

- **Types of Categorical Data**
- **1. Nominal Data:** Nominal data consists of categories without any inherent order or ranking. These are simple labels used to classify data.
- **Example:** 'Red', 'Blue', 'Green' (Car Color).
- **Encoding Options:** One-Hot Encoding or Label Encoding, depending on the model's needs.
- **2. Ordinal Data:** Ordinal data includes categories with a defined order or ranking, where the relationship between values is important.
- **Example:** 'Low', 'Medium', 'High' (Car Engine Power).
- **Encoding Options:** Ordinal Encoding.

CONTINUE

Techniques to perform Categorical Data Encoding

Categorical Data Encoding Techniques



One-Hot Encoding : Converts categories into binary columns.



Label Encoding : Assigns a unique integer to each category.



Ordinal Encoding : Maps categories to ordered integers.



Binary Encoding : Converts categories into binary digits.



Count (Frequency) Encoding : Replaces categories with their frequency or count.



Target Encoding : Replaces categories with the mean target value for each category.

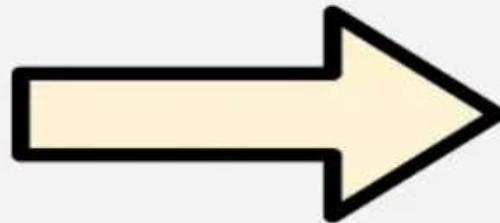


CONTINUE

- **1. Label Encoding**
- Label Encoding assigns each category a unique integer. It is simple and memory-efficient but may unintentionally imply an order among categories when none exists.
- Used in tree-based models like Decision Trees or XGBoost.
- **Pros:** Simple and memory-efficient.
- **Cons:** Introduces implicit order which may be misinterpreted by non-tree models when used with nominal data.

CONTINUE

Color
Red
Green
Blue
Red



Color
0
1
2
0



CONTINUE

- **2. One-Hot Encoding**
- **One-Hot Encoding** converts categories into binary columns with each column representing one category. It prevents false ordering but can lead to high dimensionality if there are many unique values.
- Used in linear models, logistic regression and neural networks.
- **Pros:** Does not assume order; widely supported.
- **Cons:** Can cause high dimensionality and sparse data when feature has many categories.

CONTINUE

Color		Red	Green	Blue
Red	→	1	0	0
Green		0	1	0
Blue		0	0	1
Green		0	1	0



CONTINUE

- **3. Ordinal Encoding**
- Ordinal Encoding maps categories to integers while preserving their natural order. This works well for ordered data like ratings but is not suitable for nominal variables.
- Used for ordered features like ratings or education levels.
- **Pros:** Maintains order; reduces dimensionality.
- **Cons:** Not suitable for nominal categories.

CONTINUE

Original Encoding	Ordinal Encoding
Poor	1
Good	2
Very Good	3
Excellent	4



CONTINUE

- **4. Target Encoding**
- **Target Encoding** also known as Mean Encoding is a technique where each category in a feature is replaced by the mean of the target variable for that category.
- Useful for high-cardinality features like ZIP codes or product IDs.
- **Pros:** Captures relationship to target variable.
- **Cons:** Risk of overfitting, also must apply smoothing/statistical techniques.

CONTINUE

Category	Mean Target (Experience)
Engineer	0.8
Lawyer	0.5
Nurse	0.3

Target Encoding



CONTINUE

- **5. Binary Encoding**
- **Binary encoding** represents categories as binary codes and splits them across multiple columns. It is efficient for high-cardinality data but slightly more complex to implement.
- Applied in high-cardinality text/NLP tasks to save memory.
- **Pros:** Reduces dimensionality, more memory-efficient than one-hot encoding.
- **Cons:** Slightly more complex; requires careful handling of missing values.

CONTINUE

Candidate Solution (A)

Activation function	Learning rate	Momentum	1* layer	2* layer
1 0	0 1 1 0 1 0	1 0 1 0	1 0 1 0 0	1 1 0 1 1

2 0.40625 0.625 20 27 ← decimal value

Candidate Solution (B)

Activation function	Learning rate	Momentum	1* layer	2* layer
1 1	0 0 0 0 1 0	0 0 1 0	0 0 1 0 0	0 0 0 0 0

3 0.03125 0.125 4 0 ← decimal value



CONTINUE

- **6. Frequency Encoding**
- **Frequency Encoding** assigns categories values based on how often they occur in the dataset. It is simple and compact but can introduce data leakage if applied improperly.
- Effective in retail, e-commerce or clickstream data for popularity trends.
- **Pros:** Low computational and storage requirements.
- **Cons:** Can introduce data leakage if not handled properly.

CONTINUE

Color
Red
Green
Blue
Green



Color	Frequency
Red	1
Green	2
Blue	1



CONTINUE

Differences between Various Techniques

Technique	Suitable For	Dimensionality	Overfitting Risk	Interpretability
One-Hot Encoding	Nominal	High	Low	High
Label Encoding	Ordinal (sometimes Nominal)	Low	Medium	Medium
Ordinal Encoding	Ordinal	Low	Medium	High
Binary Encoding	High-cardinality features	Medium	Medium	Medium
Frequency Encoding	High-cardinality	Low	High	Medium
Target Encoding	High-cardinality	Low	High	Low-Medium



Feature Scaling

- Feature engineering is the process of creating, transforming or selecting the most relevant variables (features) from raw data to improve model performance. Effective features help the model capture important patterns and relationships in the data. It directly contributes to model building in the following ways:
- Well-designed features allow models to learn complex patterns more effectively.
- Reduces noise and irrelevant information, improving prediction accuracy.
- Helps prevent overfitting by emphasizing meaningful data signals.
- Simplifies model interpretation by creating more informative and understandable inputs.



- **1. Absolute Maximum Scaling**
- Absolute Maximum Scaling rescales each feature by dividing all values by the maximum absolute value of that feature. This ensures the feature values fall within the range of -1 to 1. While simple and useful in some contexts, it is highly sensitive to outliers which can skew the max absolute value and negatively impact scaling quality.

CONTINUE

- Performing Absolute Maximum Scaling
- Computes max absolute value per column with `np.max(np.abs(df), axis=0)`.
- Divides each value by that max absolute to scale features between -1 and 1.
- Displays first few rows of scaled data with `scaled_df.head()`.



2. Min-Max Scaling

- Min-Max Scaling transforms features by subtracting the minimum value and dividing by the difference between the maximum and minimum values. This method maps feature values to a specified range, commonly 0 to 1, preserving the original distribution shape but is still affected by outliers due to reliance on extreme values.
- $$X_{\text{scaled}} = \frac{X_i - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}}$$
- Scales features to range.
- Sensitive to outliers because min and max can be skewed.
- **Code Example:** Performing Min-Max Scaling
- Creates MinMaxScaler object to scale features to range.
- Fits scaler to data and transforms with `scaler.fit_transform(df)`.
- Converts result to DataFrame maintaining column names.
- Shows first few scaled rows with `scaled_df.head()`.



3. Normalization (Vector Normalization)

- Normalization scales each data sample (row) such that its vector length (Euclidean norm) is 1. This focuses on the direction of data points rather than magnitude making it useful in algorithms where angle or cosine similarity is relevant, such as text classification or clustering.
- $X_{scaled} = \frac{X_i}{\|X\|}$
- Where:
- X_i is each individual value.
- $\|X\|$ represents the Euclidean norm (or length) of the vector X .
- Normalizes each sample to unit length.
- Useful for direction-based similarity metrics.



4. Standardization

- Standardization centers features by subtracting the mean and scales them by dividing by the standard deviation, transforming features to have zero mean and unit variance. This assumption of normal distribution often benefits models like linear regression, logistic regression and neural networks by improving convergence speed and stability.
- $$X_{\text{scaled}} = \frac{X_i - \mu}{\sigma}$$
- where μ = mean, σ = standard deviation.
- Produces features with mean 0 and variance 1.
- Effective for data approximately normally distributed.
-



CONTINUE

- **5. Robust Scaling**
- Robust Scaling uses the median and interquartile range (IQR) instead of the mean and standard deviation making the transformation robust to outliers and skewed distributions. It is highly suitable when the dataset contains extreme values or noise.
- $$X_{\text{scaled}} = \frac{X_i - \text{median}}{IQR}$$
- Reduces influence of outliers by centering on median
- Scales based on IQR, which captures middle 50% spread



CONTINUE

- **Advantages**
- **Improves Model Performance:** Enhances accuracy and predictive power by presenting features in comparable scales.
- **Speeds Up Convergence:** Helps gradient-based algorithms train faster and more reliably.
- **Prevents Feature Bias:** Avoids dominance of large-scale features, ensuring fair contribution from all features.
- **Increases Numerical Stability:** Reduces risks of overflow/underflow in computations.
- **Facilitates Algorithm Compatibility:** Makes data suitable for distance- and gradient-based models like SVM, KNN and neural networks.

Handling Time Series data.

ML Approaches for Time Series

Multi-Layer Perceptron (MLP)

01

Recurrent Neural Networks (RNN)

02

Convolutional Neural Networks (CNN)

03

Decision Tree-Based Models

04

Transformer Neural Networks

05



CONTINUE

- **Understanding Time Series Data**
- Time series data consists of observations collected at regular time intervals, such as daily stock prices, monthly sales figures, or yearly climate data. Key components of time series data include:
- **Trend:** The long-term increase or decrease in the data.
- **Seasonality:** Regular, repeating patterns or cycles in the data.
- **Irregularity/Noise:** Random variations that do not follow a pattern.
- **Cyclicity:** Long-term cycles that are not of a fixed period.



CONTINUE

- **Classical Methods for Time Series Forecasting**
- **Naive Model:** The naive model uses the last observed value as the forecast for the next period. It is simple but often serves as a baseline for more complex models.
- **Exponential Smoothing (ES):** Exponential smoothing methods forecast future values by averaging past observations with exponentially decreasing weights. Variants like Holt-Winters can capture trends and seasonality.
- **ARIMA/SARIMA:** ARIMA (AutoRegressive Integrated Moving Average) combines autoregression and moving averages to model time series data. SARIMA extends ARIMA by incorporating seasonal components.
- **Linear Regression:** Linear regression models the relationship between the target variable and one or more independent variables. It is straightforward but may not capture complex patterns in time series data.
- The problem with classical methods and machine learning approaches for time series forecasting lies in their limitations and the complexity of real-world data.
- **Naive Method:** Simple but often serves as a baseline for more complex models. It does not account for trends, seasonality, or other factors that can affect demand.
- **Exponential Smoothing (ES):** ES methods forecast future values by averaging past observations with exponentially decreasing weights. Variants like Holt-Winters can capture trends and seasonality.
- **ARIMA/SARIMA:** ARIMA combines autoregression and moving averages to model time series data. SARIMA extends ARIMA by incorporating seasonal components.



CONTINUE

- **Linear Regression:** Linear regression models the relationship between the target variable and one or more independent variables. It is straightforward but may not capture complex patterns in time series data.
- **Machine Learning Methods for Time Series Forecasting**
- Machine learning (ML) approaches have gained significant attention in time series forecasting due to their ability to capture complex patterns and relationships in data.
- **1. Multi-Layer Perceptron (MLP)**
- A [Multi-Layer Perceptron \(MLP\)](#) is a type of feedforward neural network composed of an input layer, one or more hidden layers, and an output layer. Each neuron in the hidden and output layers applies a weighted sum of inputs, adds a bias, and passes the result through a nonlinear activation function.
- **How it works in Time Series Forecasting?**
- **Input Representation:** $t+1$. Time series data is often transformed into a supervised learning problem by creating lagged features. For example, to forecast the next value y_{t+1} the input to the MLP could be previous values $y_t, y_{t-1}, \dots, y_{t-n}$.
- **Training Process:** The model learns weights and biases by minimizing the prediction error on training data using backpropagation and gradient descent.
- **Prediction:** Once trained, the MLP uses the learned weights to transform the input lags into the forecasted value(s).



CONTINUE

- **2. Recurrent Neural Networks (RNN)**
- Recurrent Neural Networks (RNNs) are designed to process sequences by maintaining a hidden state that captures information about previous elements in the sequence. They are particularly suited for time series data where temporal dependencies are crucial.
- **How it works in Time Series Forecasting?**
- **Sequential Processing:** At each time step t , the RNN receives an input x_t and updates its hidden state h_t based on the previous hidden state h_{t-1} and the current input.
- **Hidden State:** The hidden state acts as a memory that carries information forward through the sequence, enabling the network to capture temporal dependencies.
- **Output Generation:** The output at each time step can be used directly for forecasting or co



CONTINUE

- **3. Convolutional Neural Networks (CNN)**
- **Explanation:** Convolutional Neural Networks (CNNs) are primarily used for spatial data, such as images, but can be adapted for time series forecasting by treating the time dimension as a spatial dimension.
- **How it works in Time Series Forecasting?**
- **Convolutional Layers:** Apply filters (kernels) to the input time series data to extract local patterns. For example, a 1D convolution over the time axis can identify patterns over a fixed window of time steps.
- **Pooling Layers:** Down-sample the feature maps produced by the convolutional layers to reduce dimensionality and computational complexity.
- **Fully Connected Layers:** After convolutional and pooling layers, fully connected layers can combine extracted features to make final predictions.



CONTINUE

- **4. Decision Tree-Based Models**
- Decision Tree-based models, such as [Random Forest](#) and [Gradient Boosting](#) (e.g., LightGBM, CatBoost), are powerful techniques that combine multiple decision trees to improve prediction accuracy and handle complex, non-linear relationships.
- **How it works in Time Series Forecasting?**
- **Feature Engineering:** Time series data is transformed into a set of features (e.g., lagged values, rolling statistics) that can be used as inputs to the decision trees.
- **Random Forest:** Constructs multiple decision trees using different subsets of data and features, and averages their predictions to reduce variance and improve accuracy.
- **Gradient Boosting:** Sequentially builds decision trees where each tree corrects errors made by previous trees, using techniques like LightGBM and CatBoost to optimize for speed and performance.



CONTINUE

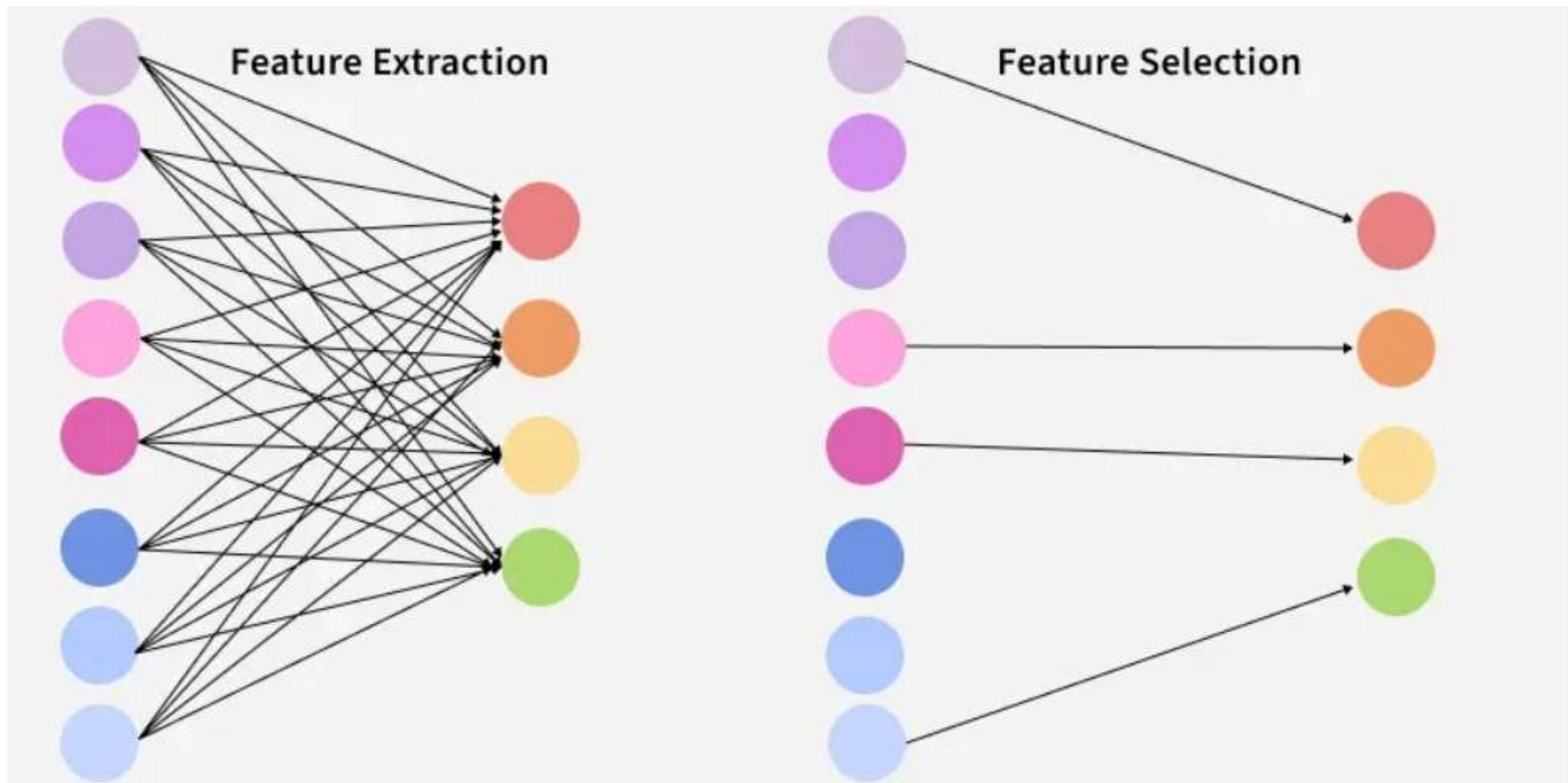
- **5. Transformer Neural Networks**
- **Transformers**, originally designed for natural language processing, use self-attention mechanisms to weigh the importance of different positions in the input sequence, enabling the model to capture long-range dependencies.
- **How it works in Time Series Forecasting?**
- **Self-Attention Mechanism:** Each position in the input sequence attends to all other positions, allowing the model to consider the entire sequence context when making predictions. The attention scores determine the relevance of each time step.
- **Encoder-Decoder Structure:** Typically used in sequence-to-sequence tasks. The encoder processes the input sequence, and the decoder generates the output sequence (forecast).
- **Positional Encoding:** Since transformers do not have inherent sequential information, positional encodings are added to the input embeddings to provide information about the relative positions of time steps.



Overview of Feature Selection and Feature Extraction Techniques

Feature selection and feature extraction are two key techniques used in machine learning to improve model performance by handling irrelevant or redundant features. While both works on data preprocessing, feature selection uses a subset of existing features whereas feature extraction transforms data into a new feature.

Continue





Continue

- **Feature selection**: Involves selecting a subset of the most relevant features that are actually contributing in prediction while discarding the rest features. This helps improve reducing overfitting and increased accuracy. Common techniques include filter, wrapper and embedded methods.
- **Feature extraction**: Transforms existing features into a new set of features that captures better underlying patterns in data. It is useful when raw data is in high dimension or complex. Techniques like PCA, LDA and Autoencoders are used for this purpose.



Continue

Feature Selection	Feature Extraction
Selects a subset of relevant features from the original set of features.	Transforms original features into a new, more informative set.
Reduces dimensionality while keeping original features.	Reduces dimensionality by transforming data into a new space.
Methods include Filter, Wrapper and Embedded techniques.	Methods include PCA, LDA, Kernel PCA and Autoencoders.
Requires domain knowledge and feature engineering.	Can be applied to raw data without prior feature engineering.
Enhances interpretability and reduces overfitting.	Improves performance and handles nonlinear relationships.
May lose useful information if important features are removed.	May introduce redundancy and noise if extracted features are not well-defined.



Data

Transformation, Data Normalization.

- Data normalization is a preprocessing method that resizes the range of feature values to a specific scale, usually between 0 and 1. It is a feature scaling technique used to transform data into a standard range. Normalization ensures that features with different scales or units contribute equally to the model and improves the performance of many machine learning algorithms.
- **Key Features of Normalization:**
- Maps the minimum and maximum of a feature to a defined range
- Preserves the relative relationships of the original data
- Useful for algorithms that rely on distance metrics such as k-Nearest Neighbours and clustering



Why do we need Normalization?

- Machine learning models often assume that all features contribute equally. Features with different scales can dominate the model's behavior if not scaled properly. Using normalization, we can:
- **Ensure Equal Contribution of Features:** Prevents features with larger scales from dominating models that are sensitive to magnitude such as K-Nearest Neighbours or neural networks.
- **Improve Model Performance:** Algorithms that rely on distances or similarities (KNN, K-Means clustering) perform better when features are normalized.
- **Accelerate Convergence:** Helps gradient-based algorithms like logistic regression or neural networks converge faster by keeping feature values in a similar range.
- **Maintain Interpretability of Scales:** By converting all features to a common range, it's easier to understand their relative impact on predictions.



Continue

Feature	Normalization (Min-Max)	Standardization (Z-score)
Goal	Rescale data to a specific range	Center data to mean 0, SD 1
Range of values	Fixed (e.g., 0–1)	Not fixed
Effect of outliers	Sensitive	Less sensitive
Assumes data distribution	No	Assumes roughly Gaussian
Use case	Distance-based algorithms	Algorithms assuming Gaussian or regularization
Example	Scaling pixel values to [0,1]	Scaling test scores to z-scores

Different Data Normalization Techniques

- **1. Min-Max Normalization**
- Min-Max normalization rescales a feature to a specific range, typically [0, 1]:
- $$X_{normalized} = \frac{X - X_{min}}{X_{max} - X_{min}}$$
- The minimum value maps to 0
- The maximum value maps to 1
- Other values are scaled proportionally



Continue

- **2. Decimal Scaling**

- Decimal scaling normalizes data by shifting the decimal point of values:

- $$v' = \frac{v}{10^j}$$

- j is the smallest integer such that the maximum absolute value of v' is less than 1

- **3. Logarithmic Transformation**

- Log transformation compresses large values and spreads out small values:
- Reduces skewness in data
- Stabilizes variance across features

Continue

- **4. Unit Vector (Vector) Normalization**
- Scales a data vector to have a magnitude of 1:
- $$X' = \frac{X}{||X||}$$
- Commonly used in text mining and machine learning algorithms like KNN
- Preserves direction but normalizes magnitude